

Collections Generic Framework

Programación JAVA.

Objetivos

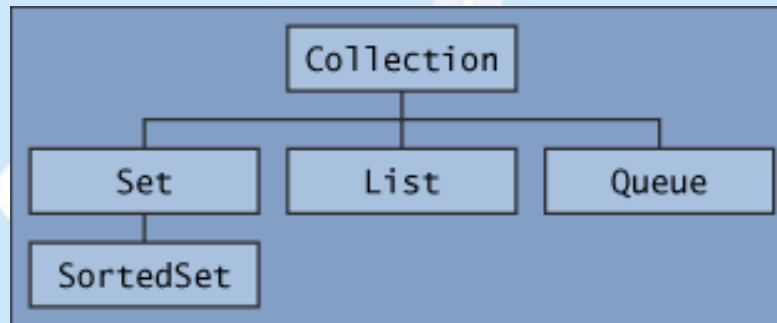
- Describir las principales interfaces e implementaciones del framework Collections
- Describir la interfaz de tipo Map
- Describir ordenación en Colecciones por medio de implementación de las interfaces Comparable y Comparator
- Uso de Colecciones genéricas
- Describir el uso de tipo de parámetros en clases genéricas
- Factorizar el código existente no genérico
- Estructura de programas para iterar sobre colecciones
- Características avanzadas en bucles

Introducción

- Colección → Objeto que es capaz de agrupar múltiples elementos en un contenedor.
- Uso → almacenamiento, recuperación, manipulación y listado de datos.
- Agrupaciones de elementos que forman un grupo homogéneo → Colección de cartas de una baraja, carpeta de correo, etc
- Framework Collection → Arquitectura para la representación y manipulación de colecciones. Todos los frameworks de Collections contienen:
 - Interfaces: Métodos para el manejo de forma uniforme
 - Implementaciones: Implementaciones concretas de las interfaces de tipo Collection
 - Algoritmos: Métodos que proporcionan algoritmos de búsqueda y ordenación. Son polimorfos → Declaración en la interfaz y uso de diversas implementaciones

API Collections

- Una colección es un objeto que es capaz de manejar un grupo de objetos → elementos. Las colecciones pueden ser de cualquier tipo
- Interface Collection: Encapsula distintos tipos de colecciones que se manejan de forma uniforme independientemente de su implementación



- API de Collections agrupa los objetos en una de las siguientes interfaces:
 - Collection: Raíz de la jerarquía, representa un grupo de objetos.
 - Interfaz denominador común que todas las implementaciones de Collection deben de implementar.
 - Uso → para proporcionar la máxima capacidad de generalización en operaciones
 - Java no proporciona implementaciones de forma directa para esta interfaz → Implementaciones para subinterfaces: List, Set, etc.

API Collections

- Set: Colección que no permite elementos duplicados. Encapsula la abstracción de un conjunto matemático.
 - Uso → representación de cartas de la baraja, procesos de una máquina, etc
- List: Colección ordenada (secuencia) en donde puede haber elementos duplicados
 - Permite obtener un control preciso sobre sus elementos. Se proporciona un acceso indexado por posición.
- Queue: colección de elementos pendientes de proceso (cola).
 - Proporciona operaciones adicionales para el manejo de colas como inserción, inspección y extracción de una cola
 - Orden normal de los elementos FIFO (First In First Out) → Puede tener otros
 - Cabeza (Head) → Elemento que se elimina en una llamada a `remove()` o `poll()`
- Deque: tipo de cola que permite la introducción y eliminación de elementos por los dos lados → Simulación de pilas y colas
- Antes de la versión de Java SE 5.0 tipo de elementos de una colección → `Object`.
- Necesario casting correcto para recuperar el elemento
- A partir de Java 5.0 se permite el uso de Colecciones genéricas

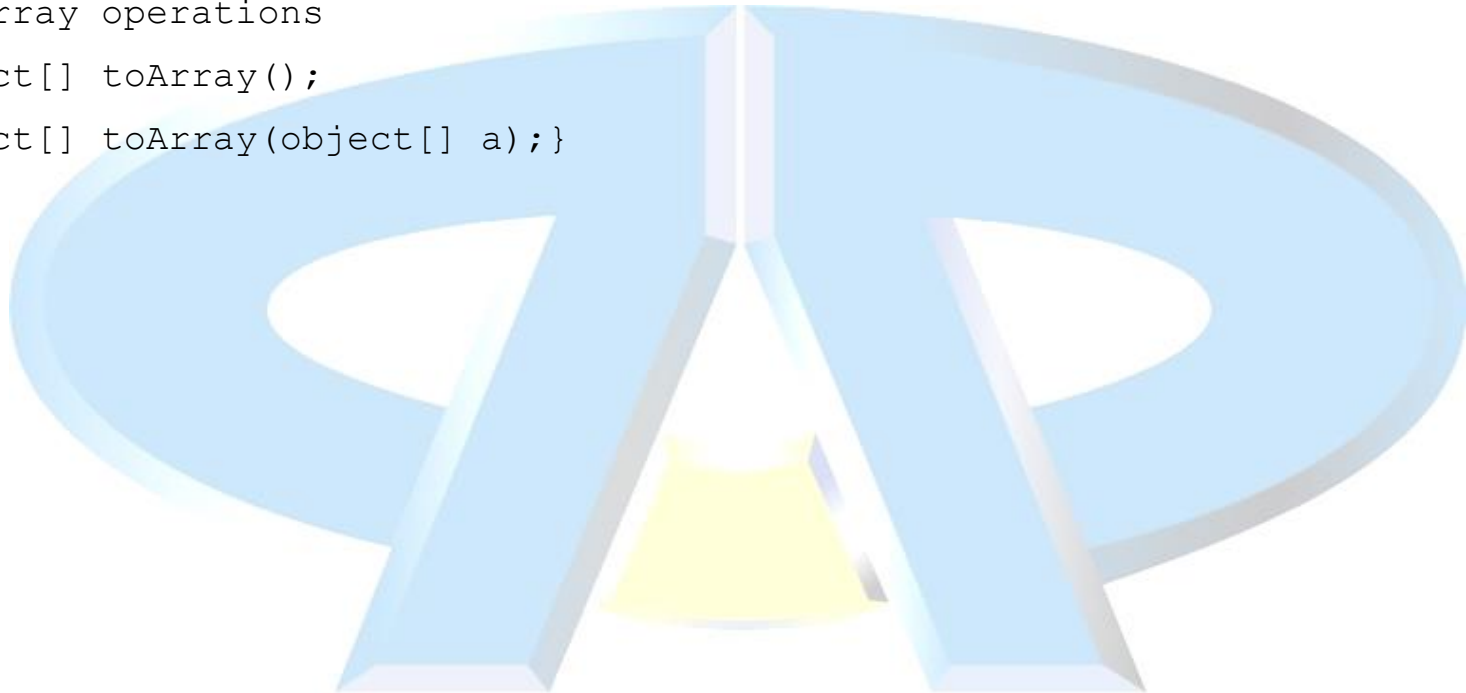
Interfaz Collection

- Proporciona la raíz de la jerarquía → mediante Up Casting se obtiene generalidad
- Interfaz:

```
public interface Collection extends Iterable<E> {  
    // Basic operations  
    int size();  
    boolean isEmpty();  
    boolean contains(Object element);  
    boolean add(E element); //optional  
    boolean remove(Object element); //optional  
    Iterator iterator();  
  
    // Bulk operations  
    boolean containsAll(Collection c);  
    boolean addAll(Collection c); //optional  
    boolean removeAll(Collection c); //optional  
    boolean retainAll(Collection c); //optional
```

Interfaz Collection

```
void clear(); //optional  
// Array operations  
Object[] toArray();  
Object[] toArray(object[] a);}
```



Interfaz Set

- No puede contener duplicados
- Representación de un conjunto matemático
- Hereda los métodos de `Collection` y añade una restricción al método `add` → Elementos no duplicados

```
public interface Set extends Collection {  
    // Basic operations  
    int size ();  
    isEmpty boolean ();  
    boolean contains (Object element);  
    boolean add (Object E); // opcional  
    boolean remove (Object element); // opcional  
    Iterator iterator();  
}
```

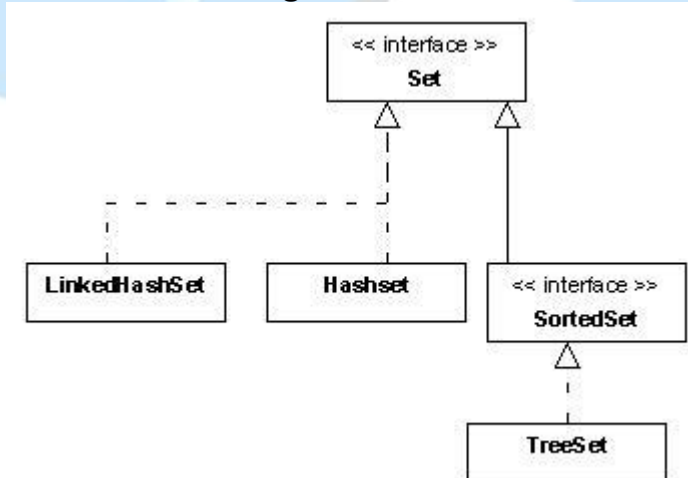

Interfaz Set

```
// Bulk operations
boolean containsAll (Collection c);
boolean addAll (Collection c); // opcional
removeAll boolean (Collection c); // opcional
boolean retainAll (Collection c); // opcional
void clear (); // opcional

// Array operations
Object [] toArray ();
Object [] toArray (Object[] a);
}
```

Interfaz Set

- Tres implementaciones de propósito general:
 - HashSet: Almacena los elementos internamente en una tabla hash
 - No se garantiza el orden de la iteración
 - Implementación con mejor rendimiento
 - TreeSet: Mas lento que un HashSet
 - Se garantiza el orden de la iteración → orden basado en sus valores
 - LinkedHashSet: Implementación con una tabla hash y una lista enlazada
 - Elementos ordenados según su orden de inserción



Interfaz List

- List: Colección ordenada (secuencia) en donde puede haber elementos duplicados.
- Permite obtener un control preciso sobre sus elementos. Se proporciona un acceso indexado por posición.
- Permite elementos duplicados
- Introduce operaciones para la gestión de:
 - Acceso por posición → manipulación de elementos basándose en su posición numérica
 - Búsqueda → búsqueda de un objeto en la lista y retorna su posición numérica
 - Iteración → se extiende el iterador para aprovechar la naturaleza secuencia de la lista
 - Range-view → realiza operaciones relativas con rangos de la lista

Interfaz List

- Clases principales que implementan la interfaz (ver API):
 - ArrayList: Array redimensionable. Implementa todas las operaciones opcionales de la interfaz. Permite inserción de elementos nulos
 - LinkedList: Implementación de una lista enlazada que permite valores nulos. Implementa todos los métodos opcionales de la interfaz List. Permite añadir/eliminar elementos por el principio y final. (Implementaciones cola / pila)
 - Vector: Clase heredada desde el JSK 1.0. Es como un ArrayList pero sus métodos están sincronizados para ser seguros con el uso de threads

Ejemplo:

```
import java.util.*;

Public class ListExample{
    public static void main(String[] args){
        List lista = new ArrayList();
        lista.add("One");
        lista.add("Second");
        lista.add(new Integer(2));
        lista.add("Second"); //Entrada duplicada, se añade
        System.out.println(lista)
    }
}

Java ListExample
[One, Second, 2, Second]
```

Interfaz Queue

- Abstracción de una cola → Colección de elementos para procesar
- Métodos adicionales a los heredado por el interfaz Collection

```
public interface Queue extends Collection {  
    Object element();  
    boolean offer(object e);  
    Object peek();  
    Object poll();  
    Object remove();  
}
```

- Se proporcionan métodos adicionales para la inserción modificación y eliminación
- Implementaciones:
 - LinkedList: Permite la implementación de una cola
 - PriorityQueue: Incorporada en Java SE 5.0. Creación de una cola con prioridad de entrada y salida

Interfaz Queue

	Lanza Excepción	Retorna un valor especial
Inserción	<code>add(e)</code>	<code>offer(e)</code>
Eliminación	<code>remove(e)</code>	<code>poll()</code>
Inspección	<code>element()</code>	<code>peek()</code>

- Ordenación típica FIFO pero no necesariamente
- Cabeza de la cola: Elemento que se elimina cuando se llama a `remove()` o `poll()`
- Fin de la cola: donde se insertan los nuevos elementos
- Método `offer(e)` inserta un elemento en la cola (si es posible). Difiere del método `add(e)` heredado de `Collection` que lanza una `unchecked exception` (`IllegalStateException`) si falla. El método `offer(e)` está designado cuando el fallo en la inserción es normal (colas de capacidad limitada)
- `remove()` y `poll()` ambos eliminan un elemento de la cabeza de la cola, difieren en que `remove()` si no hay más elementos lanza una excepción (`NoSuchElementException`)
- `element()` y `peek()` ambos retornan, pero no eliminan el elemento de la cabeza. Si la cola está vacía `element()` lanza una excepción (`NoSuchElementException`) y `peek()` devuelve `null`

Interfaz Queue

```
import java.util.*;

public class Countdown {
    public static void main(String[] args) throws InterruptedException {
        int time = Integer.parseInt(args[0]);
        Queue<Integer> queue = new LinkedList<Integer>();
        for (int i = time; i >= 0; i--) queue.add(i);
        while (!queue.isEmpty()) {
            System.out.println(queue.remove());
            Thread.sleep(1000);
        }
    }
}
```

Map interface

- Describe pares clave, valor. Por definición no puede tener elementos duplicados y una clave puede tener como máximo un valor
- Proporciona tres métodos que permite ver los mapas como colecciones.
 - `entrySet`: devuelve un conjunto (Set) que contiene los pares clave-valor
 - `KeySet`: Devuelve un conjunto (Set) con todas las claves del mapa
 - `Values`. Devuelve una colección (Collection) con todos los valores del mapa
- El interfaz Map no extiende del interfaz Collection debido a que no representa colecciones de objetos sino mapas.
- Implementaciones:
 - `HashMap`: Proporciona un mapa en donde no hay orden ni está clasificado. Es la implementación que menos coste introduce
 - `HashTable`: Clase heredada desde el JDK 1.0. Métodos sincronizados y no permite la inserción de null
 - `LinkedHashMap`: mantiene el orden de la inserción, más lenta que un `HashMap` para eliminar / añadir elementos.
 - `TreeMap`: Mapa ordenado

Map interface

```
public interface Map {  
    // Basic operations  
    put(Object key, Object value);  
    get(Object key);  
    remove(Object key);  
    boolean containsKey(Object key);  
    boolean containsValue(Object value);  
    int size();  
    boolean isEmpty();  
  
    // Bulk operations  
    void putAll(Map m);  
    void clear();  
  
    // Collection Views  
    public Set keySet();  
    public Collection values();  
    public Set entrySet();  
}
```



Map interface

- Ejemplo

```
import java.util.*;

public class TreeMapExample{

    public static void main(String[] args) {
        System.out.println("Tree Map " +
            "Example!\n");
        TreeMap tMap = new TreeMap ();
        //Adding data to a tree map
        tMap.put(1, "Sunday");
        tMap.put(2, "Monday");
        tMap.put(3, "Tuesday");
        tMap.put(4, "Wednesday");
        tMap.put(5, "Thursday");
        tMap.put(6, "Friday");
        tMap.put(7, "Saturday");
        //Retrieving all keys
        System.out.println("Keys of tree map:"
            + tMap.keySet());

        //Retrieving the value from key with
        //key number 5
        System.out.println("Key: 5 value: " +
            tMap.get(5)+ "\n");
        //Retrieving the First key and value
        System.out.println("First key: " +
            tMap.firstKey() + " Value: "
            + tMap.get(tMap.firstKey()) + "\n")
        //Retrieving the Last key and value
        System.out.println("Last key: " +
            tMap.lastKey() + " Value: "
            + tMap.get(tMap.lastKey()) + "\n");
        //Removing the first key and value
        System.out.println("Removing first " +
            data: " + tMap.remove(tMap.firstKey()
            ));
```

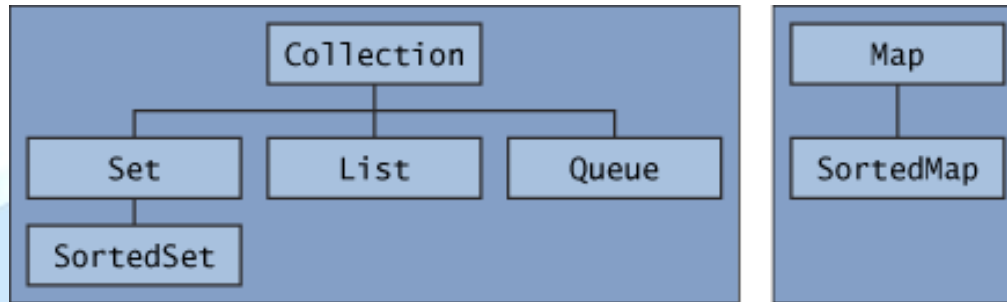
Map Interface

```
System.out.println("Now the tree map Keys: " + tMap.keySet());  
System.out.println("Now the tree map contain: " + tMap.values() + "\n");  
//Removing the last key and value  
System.out.println("Removing last data: " + tMap.remove(tMap.lastKey()));  
System.out.println("Now the tree map Keys: " + tMap.keySet());  
System.out.println("Now the tree map contain: " + tMap.values());  
  
}  
}
```

Clases Collection heredadas

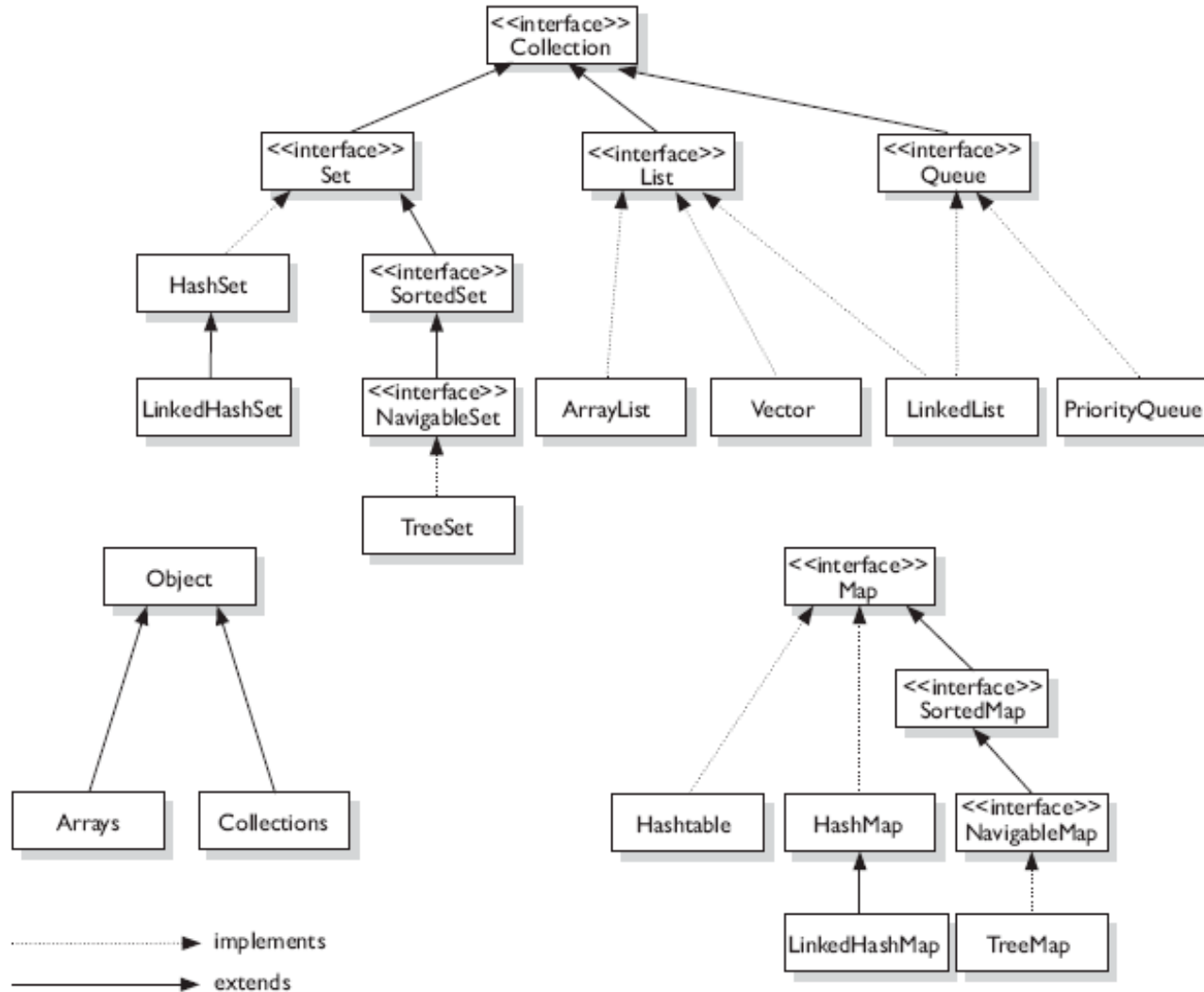
- Todavía existen clases heredadas del JDK 1.0 y 1.1 en el actual JDK pero que han sido rediseñadas para interactuar con el nuevo interfaz
- Clase Vector implementa la interfaz List
- La clase Stack es una extensión de Vector que añade las típicas operaciones sobre pilas
- Hashtable es una implementación de Map
- La clase Properties es una extensión de Hashtable que sólo emplea Strings para la clave y valor
- Cada una de estas colecciones tiene un método que retorna un objeto Enumeration → interfaz parecida pero incompatible con Iterator

Sumario de interfaces



- Interfaz **Collection** proporciona funcionalidad básica a todas las colecciones → métodos **add** y **remove**
- Subinterfaces de **Collection** → **Set**, **List**, **Queue** proporcionan colecciones más específicas.
- Interfaz **Set** no permite duplicados
- Interfaz **List** proporciona una colección ordenada → Posibilidad de acceder a los elementos de la lista por su posición
- El interfaz **Queue** añade operaciones como inserción, extracción y operaciones de inspección en una cola con orden FIFO (generalmente)
- El interfaz **Map** proporciona mapeos entre una clave y un valor

Sumario de interfaces

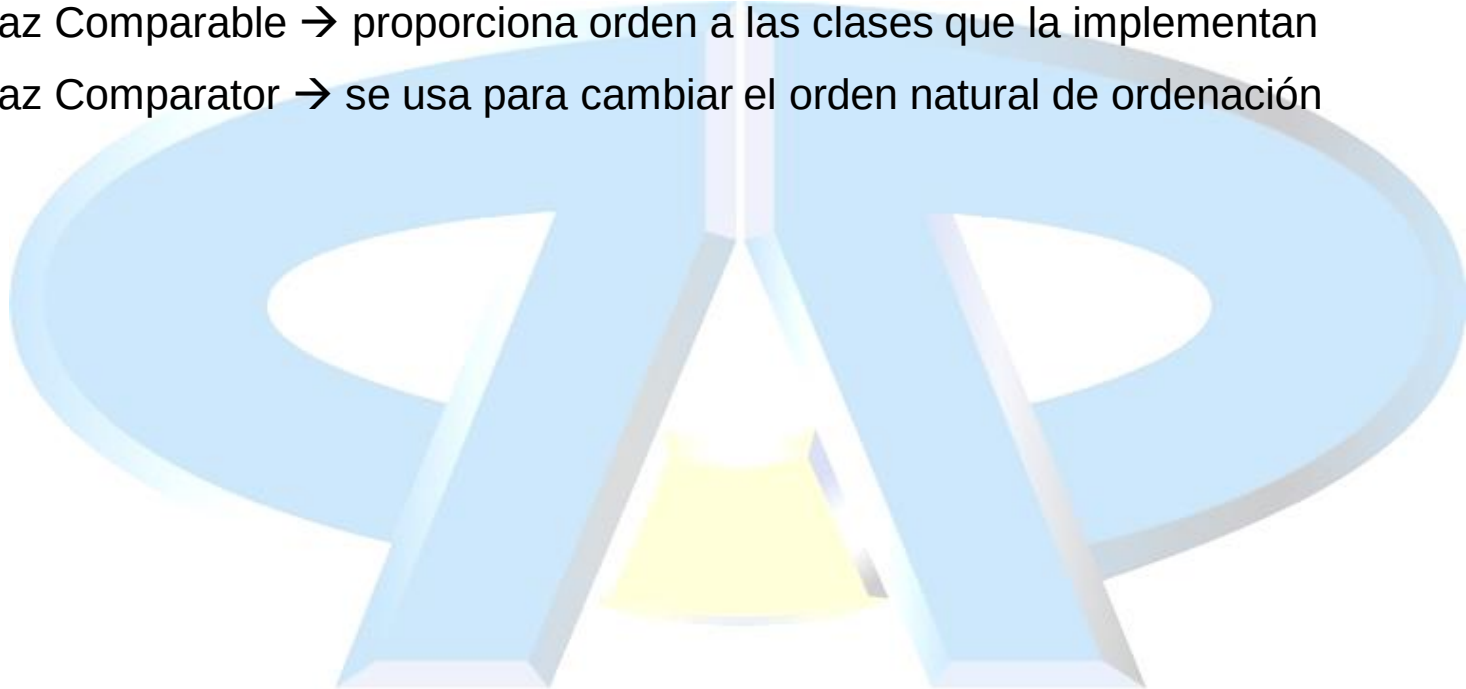


Implementaciones de Collection

	Hash Table	Resizable Array	Balanced Tree	Linked List	Hash table + Linked List
Set	HashSet		TreeSet		LinkedHashSet
List		ArrayList		LinkedList	
Deque		ArrayDeque		LinkedList	
Map	HashMap		TreeMap		LinkedHashMap

Ordenar Colecciones

- Comparación → uso de interfaces Comparable y Comparator
- Interfaz Comparable → proporciona orden a las clases que la implementan
- Interfaz Comparator → se usa para cambiar el orden natural de ordenación



Ordenar Colecciones – Interfaz Comparable

- Pertenece al paquete `java.lang`
- Declaración de una clase → orden? → Implementación interfaz comparable se proporciona orden a los objetos de la clase
- Objetivo → proporcionar orden a los objetos de una colección
- Principales objetos que implementan esta interfaz: `Byte`, `Character`, `Long`, `String`, `Date`, `Float`, `Boolean` etc
- Las colecciones puede ser ordenadas mediante la clase utilidad `java.util.Collections` (no tiene que ver con el interfaz `Collection`)
- Invocación `Collections.sort(List l)` → Si los elementos de la lista no implementan la interfaz `Comparable` → `ClassCastException` (Unchecked Exception → Runtime Exception)
- Ordenación de colección de elementos de tipo por medio de `Collections`:
 - `String` → Devuelve una lista con orden alfabético
 - `Time` → Devuelve una lista con orden cronológico
 - Numérico → Devuelve una lista en orden numérico

Ordenar Colecciones – Interfaz Comparable

- Ejemplo

```
public static void main(String[] args){  
    List listaEnteros = new ArrayList();  
    listaEnteros.add(9);  
    listaEnteros.add(2);  
    listaEnteros.add(52);  
    listaEnteros.add(22);  
    System.out.println("Lista enteros sin ordenar -->  
    "  
        + listaEnteros);  
    Collections.sort(listaEnteros);  
    System.out.println("Lista enteros ordenada --> "  
        + listaEnteros);  
}
```

Salida:

Lista enteros sin ordenar -->[9, 2, 52, 22]

Lista enteros ordenada -->[2, 9, 22, 52]

Ordenar Colecciones – Interfaz Comparable

- Clases propias → implementar interfaz Comparable
- Interfaz Comparable → Se proporciona implementación al método

```
int compareTo(Object objeto2)
```

- La implementación debe ser de la forma:
 - negativo si $\text{objeto1} < \text{objeto2}$
 - cero si $\text{objeto1} = \text{objeto2}$
 - positivo si $\text{objeto1} > \text{objeto2}$

Ordenar Colecciones – Interfaz Comparable

- Ejemplo.

```
public class Student implements Comparable{

    private String name,surname;
    int studentId=0;
    double GPA=0;

    public Student(String name,String surname,
        int studentId,double GPA){
        if (name == null ||surname == null ||
            studentId ==0 || GPA == 0)
            throw new IllegalArgumentException();
        this.name = name;
        this.surname = surname;
        this.studentId = studentId;
        this.GPA = GPA;
    }
}
```

```
public String getName(){return this.name;}
public String getSurname(){return surname;}
public int getStudentId(){return studentId;}
public double getGPA(){return this.GPA;}

//implements the compareTo method
public int compareTo(Object o){
    double f = this.GPA - ((Student)o).getGPA();
    if (f ==0){
        return 0; //Equals
    }else if (f < 0.0){
        return -1;
    }else {
        return 1;
    }
}
```

Ordenar Colecciones – Interfaz Comparable

```
import java.util.Set;
import java.util.TreeSet;
public class ComparableTest {
    public static void main(String[] args){
        Set studentSet = new TreeSet();
        studentSet.add(new Student("Jose","Perez",90,5.0));
        studentSet.add(new Student("Pablo","Conde",9,15.0));
        studentSet.add(new Student("Arturo","Vidal",10,25.0));
        studentSet.add(new Student("Miguel","Gonzalez",1,5.7));
        studentSet.add(new Student("Amalia","Suarez",2,25.4));
        Object[] arrayStudent = studentSet.toArray();
        for(Object o:arrayStudent){
            Student s = (Student)o;
            System.out.printf("Nombre %s %s ID = %d GPA =%.1f\n",
                s.getName(),s.getSurname(),s.getStudentId(),s.getGPA());
        }
    }
}
```

Ordenar Colecciones – Interfaz Comparator

- Pertenece al paquete java.util
- Ordenar objetos de una clase sin el orden natural que tienen por defecto
- Se quiere ordenar los objetos de una clase que no implementan el interfaz Comparable
- Objetivo → proporcionar orden a los objetos de una colección
- Interfaz Comparator:

```
public interface Comparator {  
    int compare(Object o1, Object o2);  
}
```
- El método `compareTo(Object o1, Object o2)` retorna:
 - Entero positivo si el primer argumento es mayor que el segundo
 - Cero si ambos argumentos son iguales
 - Entero negativo si el primer argumento es menor que el segundo
- Si cualquiera de los argumentos tiene un tipo inapropiado el método lanza la excepción `ClassCastException`.

Ordenar Colecciones – Interfaz Comparator

```
public class Student {  
  
    private String name,surname;  
    int studentId=0;  
    double GPA=0;  
  
    public Student(String name,String surname,  
        int studentId,double GPA){  
        if (name == null ||surname == null ||  
            studentId ==0 || GPA == 0)  
            throw new IllegalArgumentException();  
        this.name = name;  
        this.surname = surname;  
        this.studentId = studentId;  
        this.GPA = GPA;  
    }  
  
    public String getName(){return this.name;}  
    public String getSurname(){return surname;}  
    public int getStudentId(){return studentId;}  
    public double getGPA(){return this.GPA;}  
    }  
    //Clase comparador por nombre  
    public class NameComp implements Comparator{  
        public int compare(Object o1,Object o2){  
            return ((Student)o1).getName().  
                compareTo(((Student)o2).getName());  
        }  
    }  
    //Clase comparador por nota  
    public class GradeComp implements Comparator{  
        public int compare(Object o1,Object o2){  
            return ((Student)o1).getGPA().  
                compareTo(((Student)o2).getName());  
        }  
    }  
}
```


Ordenar Colecciones – Interfaz Comparator

```
//Clase comparador por nota
public class GradeComp implements Comparator{
    public int compare(Object o1,Object o2){
        if (((Student)o1).getGPA() ==
            ((Student)o2).getGPA()){
            return 0;
        }else if (((Student)o1).getGPA() <
            ((Student)o2).getGPA()){
            return -1;
        }else {
            return 1;
        }
    }
}

Public class ComparatorTest(){
    public static void main(String[] args){
        Comparator c = new NameComp();
        Collection studentSet = new TreeSet(c);
        studentSet.add(new Student("Jose","Perez",90,5.0));
        studentSet.add(new Student("Pablo","Conde",9,15.0));
        studentSet.add(new Student("Arturo","Vidal",10,25.0));
        studentSet.add(new Student("Miguel","Gonzalez",1,5.7));
        studentSet.add(new Student("Amalia","Suarez",2,25.4));
        Object[] arrayStudent = studentSet.toArray();
        for(Object o:arrayStudent){
            Student s = (Student)o;
            System.out.printf("Nombre %s %s ID = %d GPA
            =%.1f\n",s.getName(),s.getSurname(),
                s.getStudentId(),s.getGPA());
        }
    }
}
```


Genéricos

- Colecciones → clase que permite tener elementos de tipo Object
- Se necesitan casting explícitos → Problema de identidad de los objetos (Not type-safe)
- El framework de colecciones soporta colecciones de tipos homogéneos → un solo tipo de dato (ejemplo una colección de elementos de tipo Date)
- Solución al problema type-safe → uso de funcionalidad genérica. Se introdujo en Java SE 5.0.
- Se proporciona al compilador información del tipo usado en la colección. Se realiza una comprobación de tipos en tiempo de ejecución
- Se elimina el down casting hacia el tipo de objeto que se quiere obtener

- Código sin uso de genericos:

```
ArrayList list = new ArrayList();  
list.add(0, new Integer(1));  
int total = ((Integer)list.get(0)).intValue();//Down Casting
```

- Se recupera un objeto de tipo Integer y se hace un down casting

Genericos

```
ArrayList<Integer> list = new ArrayList<Integer>();  
list.add(0, new Integer(1));  
int total = list.get(0).intValue();
```

- Con el uso de generics se declara la List de tipo Integer y se informa al compilador del tipo de la colección
- Al obtener el valor de la posición 0, no es necesario usar clase wrapper para el tipo primitivo int, de hecho la utilidad de autoboxing permite lo siguiente:

```
ArrayList<Integer> list = new ArrayList<Integer>();  
list.add(0, 1)  
int total = list.get(0);
```

- El tipo de dato se puede incorporar en los argumentos de los métodos haciendo que sea un tipo seguro

```
void takeListOfStrings(List<String> strings) {  
    strings.add("foo"); // no problem adding a String  
}  
  
void takeListOfStrings(List<String> strings) {  
    strings.add(new Integer(42)); // NO!! strings is type safe  
}
```

Genericos

- Los tipos de retorno también se pueden ser seguros:

```
public List<Dog> getDogList() {  
    List<Dog> dogs = new ArrayList<Dog>();  
    // more code to insert dogs  
    return dogs;  
}  
}
```

- Ejemplo de uso de generics en Set:

```
public class GenSetExample{  
    public static void main(String[] args){  
        Set<String> set = new HashSet<String>();  
        set.add("One");  
        set.add("One");  
        set.add("One");  
        set.add(new Integer(2)); //Compile Error  
        Set.add("One"); //Duplicate variable, not added  
    }  
}
```

Genericos y Polimorfismo

- Polimorfismo en Collection
- `List<Integer> myList = new ArrayList<Integer>()` → Se puede asignar un ArrayList a una referencia de tipo List
- Entonces:

```
class Parent { }  
class Child extends Parent { }  
List<Parent> myList = new ArrayList<Child>(); → No funciona
```
- Regla → El tipo definido en la declaración de la variable debe de coincidir con el que se le pasa al objeto.

```
List<Object> myList = new ArrayList<JButton>(); // NO!  
List<Number> numbers = new ArrayList<Integer>(); // NO!  
// remember that Integer is a subtype of Number
```

But these are fine:

```
List<JButton> myList = new ArrayList<JButton>(); // yes  
List<Object> myList = new ArrayList<Object>(); // yes  
List<Integer> myList = new ArrayList<Integer>(); // yes
```

Genericos y Polimorfismo

- El polimorfismo se aplica sólo a tipos base y por tipo base se entiende el tipo de la clase Collection. La clase puede ser personalizada con un tipo parámetro `List<JButton>`
`List<JButton> myList = new ArrayList<JButton>();`
- List y ArrayList son el tipo base y JButton es el tipo genérico
- Un ArrayList puede asignarse a un List, pero una Collection de `<JButton>` no puede ser asignada a una Collection de `<Object>`, aunque JButton es un subtipo de Object

Métodos Genericos

- Método polimorfico → Declaración para un tipo y en tiempo de ejecución se puede referir a un subtipo

```
• abstract class Animal {  
    public abstract void checkup();  
}  
  
class Dog extends Animal {  
    public void checkup() { // implement Dog-specific code  
        System.out.println("Dog checkup");  
    }  
}  
  
class Cat extends Animal {  
    public void checkup() { // implement Cat-specific code  
        System.out.println("Cat checkup");  
    }  
}  
  
class Bird extends Animal {  
    public void checkup() { // implement Bird-specific code  
        System.out.println("Bird checkup");  
    }  
}
```

Métodos Genericos

```
public class AnimalDoctor {  
    // method takes an array of any animal subtype  
    public void checkAnimals(Animal[] animals) {  
        for(Animal a : animals) {  
            a.checkup();  
        }  
    }  
  
    public static void main(String[] args) {  
        Dog[] dogs = {new Dog(), new Dog()};  
        Cat[] cats = {new Cat(), new Cat(), new Cat()};  
        Bird[] birds = {new Bird()};  
        AnimalDoctor doc = new AnimalDoctor();  
        doc.checkAnimals(dogs); // pass the Dog[]  
        doc.checkAnimals(cats); // pass the Cat[]  
        doc.checkAnimals(birds); // pass the Bird[]  
    }  
}
```

Métodos Genericos

- Cambio de Arrays por Collection generics

```
public class AnimalDoctorGeneric {  
    // change the argument from Animal[] to ArrayList<Animal>  
    public void checkAnimals(ArrayList<Animal> animals) {  
        for (Animal a : animals) {  
            a.checkup();  
        }  
    }  
  
    public static void main(String[] args) {  
        // make ArrayLists instead of arrays for Dog, Cat, Bird  
        List<Dog> dogs = new ArrayList<Dog>();  
        dogs.add(new Dog());  
        dogs.add(new Dog());  
        List<Cat> cats = new ArrayList<Cat>();  
        cats.add(new Cat());  
        cats.add(new Cat());  
        List<Bird> birds = new ArrayList<Bird>();  
        birds.add(new Bird());  
    }  
}
```


Métodos Genericos

```
// this code is the same as the Array version
AnimalDoctorGeneric doc = new AnimalDoctorGeneric();
// this worked when we used arrays instead of ArrayLists
doc.checkAnimals(dogs); // send a List<Dog>
doc.checkAnimals(cats); // send a List<Cat>
doc.checkAnimals(birds); // send a List<Bird>
}
}
Error de compilación
javac AnimalDoctorGeneric.java
AnimalDoctorGeneric.java:51: checkAnimals(java.util.
ArrayList<Animal>) in AnimalDoctorGeneric cannot be applied to
(java.util.List<Dog>)
doc.checkAnimals(dogs);
^
AnimalDoctorGeneric.java:52: checkAnimals(java.util.
ArrayList<Animal>) in AnimalDoctorGeneric cannot be applied to
(java.util.List<Cat>)
doc.checkAnimals(cats);
^
AnimalDoctorGeneric.java:53: checkAnimals(java.util.
ArrayList<Animal>) in AnimalDoctorGeneric cannot be applied to
(java.util.List<Bird>)
doc.checkAnimals(birds);
```

Métodos Genericos

- Motivo → El compilador no deja pasar un `ArrayList<Dog>` a un método que tiene como parámetro de la colección `ArrayList<Animal>`.
- Si no se produjese error → se podría introducir en la colección cualquier animal de la jerarquía.
- El compilador no sabría como comunicar que se introduce un perro en una lista que sólo es de gatos
- Solución: Cambiar la signature del método para poder indicar al compilador que se le puede pasar cualquier tipo subtipo genérico del declarado en el argumento. Esto se consigue mediante el mecanismo del uso del comodín ?

- De

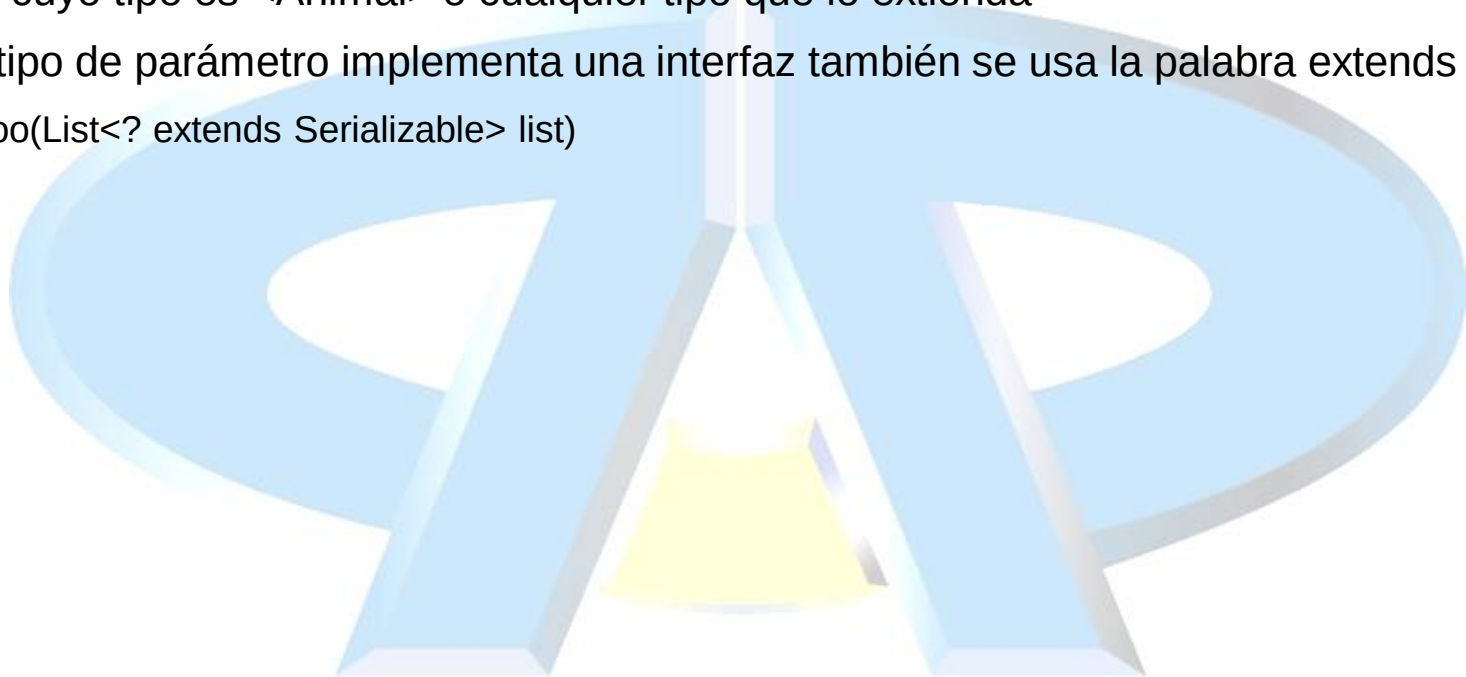
```
public void checkAnimals(List<Animal> animals)
```

- a

```
public void checkAnimals(List<? extends Animal> animals)//Correcto
```

Métodos Genericos

- `<? extends Animal>` con esta declaración se dice “Puedo asignar una colección subtipo de List y cuyo tipo es `<Animal>` o cualquier tipo que lo extienda
- Si el tipo de parámetro implementa una interfaz también se usa la palabra `extends`
- `void foo(List<? extends Serializable> list)`



Declaraciones genéricas

- Se pueden definir parámetros en la declaración de Interfaces y clases

```
public interface List<E>
```

- <E> es un hueco que debe de ser rellenado con el tipo de parámetro a pasar. El interfaz List se comporta como una plantilla genérica. Se le puede pasar: List<Dog> o List<Integer>

- Métodos: Se puede declarar el tipo de parámetro de la interfaz:

```
boolean add(E o)
```

- Clases genéricas: permiten su parametrización por medio de tipos genéricos de parámetros

- Public class A <T> {

- Métodos genéricos → los métodos de una clase pueden ser genéricos sin serlo la clase:

```
import java.util.*;  
public class CreateAnArrayList {  
    public <T> void makeArrayList(T t) {  
        // take an object of an unknown type and use a "T" to represent the type  
        List<T> list = new ArrayList<T>(); // now we can create the list using "T"  
        list.add(t);  
    }  
}
```

Declaraciones genéricas

```
public class Rental {  
    private List rentalPool;  
    private int maxNum;  
    public Rental(int maxNum, List rentalPool) {  
        this.maxNum = maxNum;  
        this.rentalPool = rentalPool;  
    }  
    public Object getRental() {  
        // blocks until there's something available  
        return rentalPool.get(0);  
    }  
    public void returnRental(Object o) {  
        rentalPool.add(o);  
    }  
}
```

Declaraciones genéricas

- Se quiere una subclase de Alquiler para el alquiler de coches.

```
import java.util.*;

public class CarRental extends Rental {
    public CarRental(int maxNum, List<Car> rentalPool) {
        super(maxNum, rentalPool);
    }
    public Car getRental() {
        return (Car) super.getRental();
    }
    public void returnRental(Car c) {
        super.returnRental(c);
    }
    public void returnRental(Object o) {
        if (o instanceof Car) {
            super.returnRental(o);
        } else {
            System.out.println("Cannot add a non-Car");
            // probably throw an exception
        }
    }
}
```

Declaraciones genéricas

- Problemas:
 1. Método returnRentail() se hace un chequeo del tipo de objeto ya que acepta como parámetro un Object
 2. No se puede cambiar el tipo de parámetro del método returnRentail(Object o) para que tome un objeto de tipo Car
 3. No se quieren hacer subclases de cada una de las variantes de tipo de alquiler (coches, ordenadores, etc)
- Solución: Clase con tipos genéricos

Declaraciones genéricas

```
import java.util.*;

public class RentalGeneric<T> {
    // "T" is for the type parameter
    private List<T> rentalPool;
    // Use the class type for the
    // List type
    private int maxNum;
    public RentalGeneric(
        int maxNum, List<T> rentalPool) {
        // constructor takes a
        // List of the class type
        this.maxNum = maxNum;
        this.rentalPool = rentalPool;
    }

    public T getRental() {
        // we rent out a T
        // blocks until there's something available
        return rentalPool.get(0);
    }

    public void returnRental(T returnedThing) {
        // and the renter
        // returns a T
        rentalPool.add(returnedThing);
    }
}
```


Genéricos: Código heredado no genérico

- Convivencia de código genérico y código no genérico (anterior a Java se 1.5)
- Tipos genéricos sin tipo de argumento → raw types

Si no se especifica un tipo de parámetro en los objetos de tipos Collection → warning

```
public class TestBadLegacy {
    public static void main(String[] args) {
        List<Integer> myList = new ArrayList<Integer>();
        myList.add(4);
        myList.add(6);
        Inserter in = new Inserter();
        in.insert(myList); // pass List<Integer> to legacy code
    }
}

class Inserter {
    // method with a non-generic List argument
    void insert(List list) {
        list.add(new Integer(42)); // adds to the incoming list
    }
}
```

Iteradores

- Interfaz Iterator: Objeto que permite recorrer una colección permitiendo el borrado de elementos.
- Obtención de un objeto de tipo iterador para una colección: método iterator()
- Interfaz de Iterator:

```
public interface Iterator<E> {  
    boolean hasNext();  
    E next();  
    void remove(); //optional  
}
```
- Métodos de Iterator:
 - hasNext() → se usa para saber si hay más elementos
 - next() → Proporciona el siguiente elemento
 - remove() → Elimina el elemento actual. Invocación después de next y sólo una vez → en otro caso lanzamiento de excepción java.lang. IllegalStateException

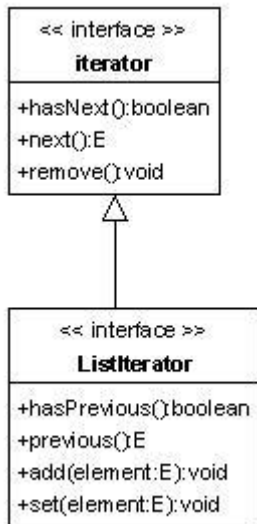
•

Iteradores

```
public class Test {  
    public static void main(String[] args){  
        Collection<String> c = new ArrayList<String>();  
        c.add("uno");  
        c.add("dos");  
        c.add("tres");  
        c.add("cuatro");  
        c.add("cinco");  
        System.out.println("Antes: " + c);  
        Iterator it = c.iterator();  
        while(it.hasNext()){  
            it.next();  
            it.remove();  
        }  
        System.out.println("Despues: " + c);  
    }  
}
```

Iteradores

- ListIterator: permite un recorrido de la colección avanzando y retrocediendo
- Método set: después de su invocación cambia el elemento actual
- ListIterator: permite un recorrido de la colección avanzando y retrocediendo
- Método set: después de su invocación cambia el elemento actual



```
public void deleteAll(Collection<NameList> c){
    for(Iterator<NameList> i=c.iterator;i.hasNext();){
        NameList n11 = i.next();
        n11.deleteItem();
    }
}
```

Bucle mejorado:

```
public void deleteAll(Collection<NameList> c){
    for(NameList n : c){
        n.deleteItem();
    }
}
```

Expresiones Lambda

- En la versión 8 de JSE se añaden las expresiones lambda, lo que va a aportar un acercamiento a la programación funcional, haciendo que las funciones se comporten como entidades de primer nivel como los valores primitivos o los objetos.
- Esto implicará que se podrá pasar funciones como valores de variables, valores de retorno de los métodos o parámetros, en tiempo de ejecución.
- Ejemplo:
 - Supongamos una clase Coche
 - A partir de un listado de objetos de tipo Coche, es necesario realizar un filtrado de acuerdo al color del mismo
 - Se podría implementar un método como el siguiente:

Coche
- color: String
- potencia: int
- numPuertas: int
- tipoCambio: String

Expresiones Lambda

```
public static List<Coche> filtrar(List<Coche> lista, String color) {  
    List<Coche> resultado = new ArrayList<Coche>();  
    for (Coche coche : lista) {  
        if (coche.getColor().equals(color)) {  
            resultado.add(coche);  
        }  
    }  
    return resultado;  
}
```

- ¿Pero qué pasaría si se necesitase filtrar por otra característica, como la potencia o el número de puertas?
- Se podría encapsular el filtro en un interface FiltroCoche, que se pasaría de parámetro al método filtrar y que tendría tantas implementaciones como tipos de filtro distintos:

- FiltroCocheColor
- FiltroCochePotencia
- FiltroCocheColorPotencia

<<interface>>

FiltroCoche

+ filtro(Coche): boolean

Expresiones Lambda

- Pero la implementación de los distintos filtros sería muy similar, y se necesitarían múltiples implementaciones.
 - Una alternativa es el uso de expresiones lambda
 - Una expresión lambda se compone de tres elementos:
 - Un listado de parámetros separados por comas y encerrados en paréntesis. Si solo es uno no es necesario.
 - El símbolo de flecha hacia la derecha: ->
 - Un cuerpo que puede ser un bloque de código encerrado entre llaves o una sola expresión.
- (parámetros) -> {cuerpo}
- Por ejemplo:
 - `(int a, int b) -> a + b`
 - `int a -> a + 1`
 - `(int a, int b) -> { System.out.println(a + b); return a + b; }`
 - `() -> new ArrayList()`

Expresiones Lambda

- Otra forma de expresión lambda es la referencia a métodos:
 - Referencia::método
 - Por ejemplo
 - `File::canRead` equivale a `File f -> f.canRead();`
- Las expresiones lambda son funciones anónimas y se pueden utilizar donde el tipo aceptado sea un interfaz funcional
- Interfaz funcional:
 - Es un interface con un único método abstracto
 - La expresión lambda proporciona la implementación de ese método
 - Puede estar anotada con `@FunctionalInterface`
- Para la utilización de las funciones lambda, se han creado nuevas interfaces funcionales , como por ejemplo las contenidas en `java.util.stream`, que permite la programación funcional sobre flujos de valores.

Tipos de Expresiones Lambda

- Consumer – Consumidor
 - Expresiones que aceptan un valor y no devuelven ninguno
 - `String mensaje -> System.out.println(mensaje);`
- BiConsumer – Biconsumidor
 - Expresiones que aceptan dos valores y no devuelven ninguno
 - `(String clave, int valor) -> insertar(clave, valor);`
- Supplier – Proveedor
 - Expresiones que no reciben ningún dato y devuelven un resultado
 - `() -> generarAleatorio();`
- Function – Función
 - Expresiones que reciben un valor y generan un valor, no necesariamente del mismo tipo
 - `Coche c -> c.getColor();`

Tipos de Expresiones Lambda

- BiFunction – Bifunción
 - Expresiones que reciben dos valores y generan un valor, no necesariamente del mismo tipo
 - (String nombre, int edad) -> new Persona(nombre, edad);
- UnaryOperator – OperadorUnario
 - Tipo especial de función, el operando y el resultado son del mismo tipo
 - String mensaje -> mensaje.toUpperCase();
- BinaryOperator – OperadorBinario
 - Tipo especial de bifunción, en que los dos operandos y el resultado son del mismo tipo
 - (String mensaje 1, String mensaje 2) -> mensaje1.concat(mensaje2);
- Predicate – Predicado
 - Tipo especial de función en que el resultado es de tipo boolean
 - String mensaje -> mensaje.length() > 20;

Expresiones Lambda

- En las interfaces del API de colecciones aparecen nuevos métodos para utilizar con streams y expresiones lambda.
 - Interface Collection
 - default boolean removeIf(Predicate<? super E> filter)
 - Elimina todos los elementos de la colección que satisfacen el predicado
 - default Stream<E> stream()
 - Devuelve un stream secuencial con la colección como origen
 - Interface Map
 - default void forEach(BiConsumer<? super K,? super V> action)
 - Realiza la acción para cada entrada del mapa
 - default void replaceAll(BiFunction<? super K,? super V,? extends V> function)
 - Sustituye cada valor de cada entrada por el valor resultante de aplicar la función a esa entrada
 - ...

Interface Stream

- El paquete `java.util.stream` contiene un conjunto de interfaces que permiten la aplicación de expresiones lambda sobre colecciones
 - `BaseStream`: Interface base para los streams, que son secuencias de elementos que soportan operaciones
 - `Stream`: Interface que representa una secuencia de elementos que soportan operaciones
 - `IntStream`, `DoubleStream`, `LongStream`: Interfaces que representan secuencias de elementos primitivos que soportan operaciones

Ejemplo

- Volviendo al ejemplo original de filtros de coches, la implementación de los diferentes filtros se simplifica:

```
- lista.stream().filter(c->c.color.equals("Rojo")).forEach(c->System.out.println(c));  
- lista.stream().filter(c->c.color.equals("Azul")).forEach(c->System.out.println(c));  
- lista.stream().filter(c->c.potencia > 105).forEach(c->System.out.println(c));  
- lista.stream().filter(c->c.potencia < 110).forEach(c->System.out.println(c));
```

- Como se observa en el ejemplo, la mayoría de los métodos de la clase stream devuelven a su vez streams, por lo que se pueden encadenar las llamadas a los métodos

```
int sum = widgets.stream()  
    .filter(w -> w.getColor() == RED)  
    .mapToInt(w -> w.getWeight())  
    .sum();
```

Colecciones inmutables

- Desde la versión 9 se han añadido una serie de métodos que permiten trabajar con colecciones que no permiten la modificación de sus elementos

- Método estático **of()**

- Permite crear colecciones inmutables ya inicializadas

List<Integer> lista = List.of(1, 3, 5);

- Método estático **copyOf()**

- Permite realizar copias inmutables de colecciones

List<Integer> copia = List.copyOf(lista);